Criterion C: Development

Techniques Used

1. Flutter Framework

- Provides Stateful and Stateless Widgets used to display dynamic and static elements respectively on the device's Graphical User Interface (GUI)
- Provides input and output features for the application
- Cross-platform single codebase compiles natively to both Android and iOS

2. Dart Programming Language

- Creation of custom objects (Reminder, HiveHelper, NotificationService, AnalyticsHelper) to accept input and display output data
- Processes data input by the user to ensure data saved in the database is valid
- Asynchronous functions using Future and await to retrieve data from Hive without blocking the UI thread

3. Hive NoSQL Database

- Stores data as typed Hive objects using @HiveType and @HiveField annotations
- Create, read, update and delete operations on persistent local data via the HiveHelper class
- Cross-platform offline-first local storage, data persists on-device with no cloud dependency
- TypeAdapter system allows strongly-typed Dart objects to be serialised to and from the Hive binary format

4. Visual Studio Code Editor

- Writing and compilation of Dart code using the Dart and Flutter VS Code extensions
- Built-in debugging module and debug console used to resolve bugs during development using debugPrint
- Hot-reload and hot-restart used to update the running emulator without a full rebuild
- Booting of the application on a Desktop Emulator

Data Structures Used

Data Structure	Description
List<E>[1]	An ordered and iterable collection that can be resized dynamically. It is used to store Reminder objects returned from Hive, free-slot results, and grouped calendar event data
HiveList<E>	A lazy, type-safe container provided by the Hive package. Stores Reminder objects in remindersBox and credentials in authBox, supporting fast keyed access and iteration without loading the whole dataset into memory.

Table 3.1: Data Structures Used

[1] List class - dart:core library - Dart API, accessed September 2026, https://api.flutter.dev/flutter/dart-core/List-class.html

Flutter Library Imports

Type / Package	Methods and Classes Used
Flutter Material Package package:flutter/material.dart	AsyncSnapshot, BottomNavigationBar, BottomNavigationBarItem, BuildContext, Center, CircularProgressIndicator, Column, Container, Dismissible, DropdownButton, DropdownMenuItem, ElevatedButton, Expanded, FloatingActionButton, Form, FormState, FutureBuilder, GestureDetector, GlobalKey, Icon, IconButton, InputDecoration, Key, ListTile, ListView, MaterialApp, Padding, Row, Scaffold, setState, SizedBox, StatefulWidget, StatelessWidget, State<E>, Text, TextEditingController, TextField, TextStyle, WidgetsFlutterBinding
Hive Package package:hive/hive.dart package:hive_flutter/hive_flutter.dart	Box<E>, HiveField, HiveObject, HiveType, Hive.openBox, Hive.initFlutter, TypeAdapter, registerAdapter
Flutter Local Notifications Package package:flutter_local_notifications/flutter_local_notifications.dart	AndroidInitializationSettings, AndroidNotificationDetails, FlutterLocalNotificationsPlugin, InitializationSettings, IOSInitializationSettings, IOSNotificationDetails, NotificationDetails, zonedSchedule
Timezone Package package:timezone/timezone.dart package:timezone/data/latest.dart	TZDateTime, local, initializeTimeZones
fl_chart Package package:fl_chart/fl_chart.dart	PieChart, PieChartData, PieChartSectionData
TableCalendar Package package:table_calendar/table_calendar.dart	TableCalendar, CalendarFormat, CalendarBuilders, EventLoader
Path / Provider package:path_provider/path_provider.dart package:path/path.dart	getApplicationDocumentsDirectory, join
Dart Core / Async dart:async, dart:core	DateTime, Duration, Future<T>, List<E>, Map<K,V>, String, int, double, bool

Table 3.2: Flutter Library Imports

Code Structure

Due to the large number of files in the project, only the folders and files directly relevant to the application's functionality are described in Table 3.3 below.

Folder / File	Purpose
lib/	This contans main.dart that which initialises Hive and registers TypeAdapters, configures the timezone database, and sets up the BottomNavigationBar routing
lib/models/reminder.dart	Defines the Reminder class annotated with @HiveType. Each field carries a unique @HiveField index so Hive can serialise and deserialise the object. The TypeAdapter is generated by build_runner.
lib/helpers/hive_helper.dart	Encapsulates all CRUD operations on remindersBox and authBox. Every screen and service calls this class to read or write data, keeping persistence logic separate from the UI.
lib/services/notification_service.dart	Wraps FlutterLocalNotificationsPlugin. Schedules a notification at the reminder's exact date/time using zonedSchedule(), and cancels notifications when a reminder is deleted.
lib/helpers/analytics_helper.dart	Pure Dart logic for computing analytics: groupByPriority(), completionPercentage(), and detectFreeSlots().
lib/screens/home.dart	Renders the TableCalendar widget and a daily reminder ListView filtered by the selected date.
lib/screens/add_reminder.dart	Reminder creation and editing form: TextField, DatePicker, TimePicker, colour swatches, priority/sound dropdowns, and a collapsible notes field.
lib/screens/reminder_list.dart	Displays all reminders sorted by date and time in Dismissible cards, supporting swipe-to-delete and tap-to-edit.
lib/screens/analytics.dart	Consumes AnalyticsHelper to render the priority PieChart, completion ring, and free-slot suggestion cards.
assets/fonts/	Custom font files (TTF) referenced in pubspec.yaml, applied consistently across all screens.
pubspec.yaml	Project manifest declaring all third-party dependencies, the assets/fonts path, and the minimum Flutter SDK version.

Table 3.3: Code Structure

Dart File	Packages Used	Functionalities
main.dart	material.dart hive_flutter	Initialises Hive and registers TypeAdapters. Sets up the BottomNavigationBar to route between home, add_reminder, reminder_list and analytics screens.
home.dart	material.dart table_calendar	Reads all reminders from HiveHelper and groups them by date to populate calendar markers. Filters reminders for the tapped date into a ListView.
reminder_list.dart	material.dart	Displays all reminders sorted ascending by date/time in Dismissible cards. Swipe-left deletes the reminder, cancels its notification and removes its Calendar event.
analytics.dart	material.dart fl_chart	Calls AnalyticsHelper methods and displays the priority PieChart, completion ring, and free-slot suggestion cards.
hive_helper.dart	hive.dart hive_flutter	Singleton providing all CRUD operations: addReminder(), getAllReminders(), updateReminder(), deleteReminder(), saveCredentials(), getCredentials().
notification_service.dart	flutter_local_notifications timezone	Initialises the notifications plugin. Provides scheduleNotification() and cancelNotification().
analytics_helper.dart	dart:core	Pure logic: groupByPriority() → Map<String,int>, completionPercentage() → double, detectFreeSlots() → List<FreeSlot>.
reminder.dart (reminder.g.dart)	hive.dart	Defines the Reminder data model with @HiveType/@HiveField annotations. The generated .g.dart file contains the TypeAdapter used for serialisation.

Table 3.4: Code Structure Functionalities

Reminder Data Model (reminder.dart)

CFS 2a — User can add a reminder with name, date, time, colour label, sound, and priority level

The Reminder class is the single data model for FocusGrid. It is annotated with @HiveType so the Hive package can serialise it to and from its binary on-device storage format. Each field carries a unique @HiveField index so that fields can be added or reordered in future without corrupting existing data. The structure of the Reminder object in Hive's remindersBox is shown in Table 3.5.

Field	Type	Description
id	int (key)	Auto-incremented unique key used to identify and cancel the matching local notification.
date	DateTime	The scheduled date of the reminder. Groups calendar dot markers and filters the daily list view.
time	String	The scheduled time stored as a HH:mm string. Combined with date to schedule the notification and set the Calendar event start time.
colourHex	String	Six-character hex colour selected from the colour swatch. Drives the card colour bar and the calendar dot marker.
priority	String	'Low', 'Medium' or 'High'. Populates the priority pie chart and the badge on reminder cards.
sound	String	Identifier of the notification sound chosen from the dropdown.
notes	String	Optional descriptive notes. null when the collapsible notes section is left empty.
isDone	bool	Tracks whether the reminder has been marked complete. Feeds the completion percentage on the analytics screen.

Table 3.5: Reminder Object Structure (remindersBox)

```
@HiveType(typeId: 0)
class Reminder extends HiveObject {
  @HiveField(0) late int id;
  @HiveField(1) late String name;
  @HiveField(2) late DateTime date;
  @HiveField(3) late String time;
  @HiveField(4) late String colourHex;
  @HiveField(5) late String priority;
  @HiveField(6) late String sound;
  @HiveField(7) String? notes;        // nullable — collapsible notes field
  @HiveField(8) bool isDone = false;  // defaults to not done
  @HiveField(9) String? googleEventId; // nullable until synced

  Reminder({
    required this.id,
    required this.name,
    required this.date,
    required this.time,
    required this.colourHex,
    required this.priority,
    required this.sound,
    this.notes,
    this.isDone = false,

  });
}
```

Code Snippet 3.1: Reminder Class Definition

extends HiveObject means each Reminder is self-aware of its position in the box: calling reminder.save() or reminder.delete() updates the record in place without needing its key. The late keyword signals that these non-nullable fields will always be provided at construction time.

HiveHelper Class (hive_helper.dart)

CFS 2a, 2d, 2e, 4d, 5a, 5b, 5c

HiveHelper is implemented as a singleton so that every screen and service accesses the same open Hive boxes throughout the application's lifetime. The class manages two boxes: remindersBox for all Reminder objects and authBox for OAuth 2.0 credentials.

```
class HiveHelper {
  // Private constructor prevents external instantiation
  HiveHelper._privateConstructor();
  static final HiveHelper instance = HiveHelper._privateConstructor();

  Box<Reminder>? _remindersBox;
  Box<dynamic>? _authBox;

  Future<void> init() async {
    // Hive.initFlutter() is called once in main.dart before runApp()
    Hive.registerAdapter(ReminderAdapter()); // generated by build_runner
    _remindersBox = await Hive.openBox<Reminder>('remindersBox');
    _authBox = await Hive.openBox('authBox');
  }

  Box<Reminder> get remindersBox => _remindersBox!;
  Box<dynamic> get authBox => _authBox!;
}
```

Code Snippet 3.2: HiveHelper Singleton Initialisation

Code Snippet 3.3 shows the four CRUD methods that operate on remindersBox. Each method is marked async because Hive box operations return Futures.

```
// CREATE – saves a new Reminder to the box
Future<void> addReminder(Reminder reminder) async {
  await remindersBox.put(reminder.id, reminder);
}

// READ – retrieves all stored reminders as a typed List
List<Reminder> getAllReminders() {
  return remindersBox.values.toList();
}

// UPDATE – overwrites an existing record identified by id
Future<void> updateReminder(Reminder reminder) async {
  await remindersBox.put(reminder.id, reminder);
}

// DELETE – removes a reminder from the box by its integer key
Future<void> deleteReminder(int id) async {
  await remindersBox.delete(id);
}
```

Code Snippet 3.3: HiveHelper CRUD Methods

Method	Function
addReminder(Reminder reminder)	Persists a new Reminder into remindersBox using its id as the key. Called from add_reminder.dart after validation passes.
getAllReminders()	Returns all Reminder objects as a typed List<Reminder>. Called by home.dart, reminder_list.dart and analytics_helper.dart.
updateReminder(Reminder reminder)	Overwrites the existing record with the same id key. Called when the user saves edits to an existing reminder.
deleteReminder(int id)	Removes the Reminder with the given key. Called from reminder_list.dart when a card is swiped away.

Table 3.6: Explanation of HiveHelper CRUD Methods

NotificationService (notification_service.dart)

CFS 6a, 6b

The NotificationService class wraps the FlutterLocalNotificationsPlugin and the timezone package to schedule device push notifications at the exact date and time of each reminder.

```
class NotificationService {
  static final NotificationService instance = NotificationService._();
  NotificationService._();

  final FlutterLocalNotificationsPlugin _plugin =
      FlutterLocalNotificationsPlugin();

  Future<void> initialize() async {
    // Android: display the app launcher icon in the notification
    const android = AndroidInitializationSettings('@mipmap/ic_launcher');
    // iOS: request permission to show alerts, badges and sounds
    final ios = IOSInitializationSettings(
      requestAlertPermission: true,
      requestBadgePermission: true,
      requestSoundPermission: true,
    );
    await _plugin.initialize(
      InitializationSettings(android: android, iOS: ios),
    );
  }
}
```

Code Snippet 3.7: NotificationService Initialisation

Code Snippet 3.8 shows the scheduleNotification() method. The timezone package converts the reminder's date and time into a TZDateTime object, required by zonedSchedule() to fire the notification at the correct local time even if the device's timezone changes.

```
// CFS 7a — schedules a notification at the reminder's exact date/time
Future<void> scheduleNotification(Reminder reminder) async {
  final parts = reminder.time.split(':');
  final tzDate = TZDateTime(
    local,
    reminder.date.year, reminder.date.month, reminder.date.day,
    int.parse(parts[0]), int.parse(parts[1]),
  );
  const androidDetails = AndroidNotificationDetails(
    'focusgrid_channel', 'FocusGrid Reminders',
    importance: Importance.max,
    priority: Priority.high,
  );
  const iosDetails = IOSNotificationDetails();
  const details = NotificationDetails(android: androidDetails, iOS: iosDetails);

  await _plugin.zonedSchedule(
    reminder.id,                              // matches reminder.id
    reminder.name,                             // CFS 7b — title
    reminder.notes ?? 'Tap to open FocusGrid.', // CFS 7b — body
    tzDate,
    details,
    androidAllowWhileIdle: true,
    uiLocalNotificationDateInterpretation:
        UILocalNotificationDateInterpretation.absoluteTime,
  );
}

// Cancels a previously scheduled notification when a reminder is deleted
Future<void> cancelNotification(int id) async {
  await _plugin.cancel(id);
}
```

Code Snippet 3.8: NotificationService — scheduleNotification()

The notification id is set to reminder.id, so the exact same key can later be used to cancel it in cancelNotification().

Method	Function
initialize()	Configures FlutterLocalNotificationsPlugin for Android and iOS. Called once in main.dart before runApp().
scheduleNotification(Reminder reminder)	Converts the reminder's date/time into a TZDateTime and calls zonedSchedule() to fire a push notification at that moment.
cancelNotification(int id)	Cancels a previously scheduled notification when the reminder is deleted or updated.

Table 3.8: Explanation of NotificationService Methods

add_reminder.dart File

CFS 2a, 2b, 2c, 2d, 5a, 5c, 7a

The add_reminder.dart screen is used for both creating new reminders and editing existing ones. When navigated to from reminder_list.dart with an existing reminder passed as an argument, all form fields are pre-populated with that reminder's current data.

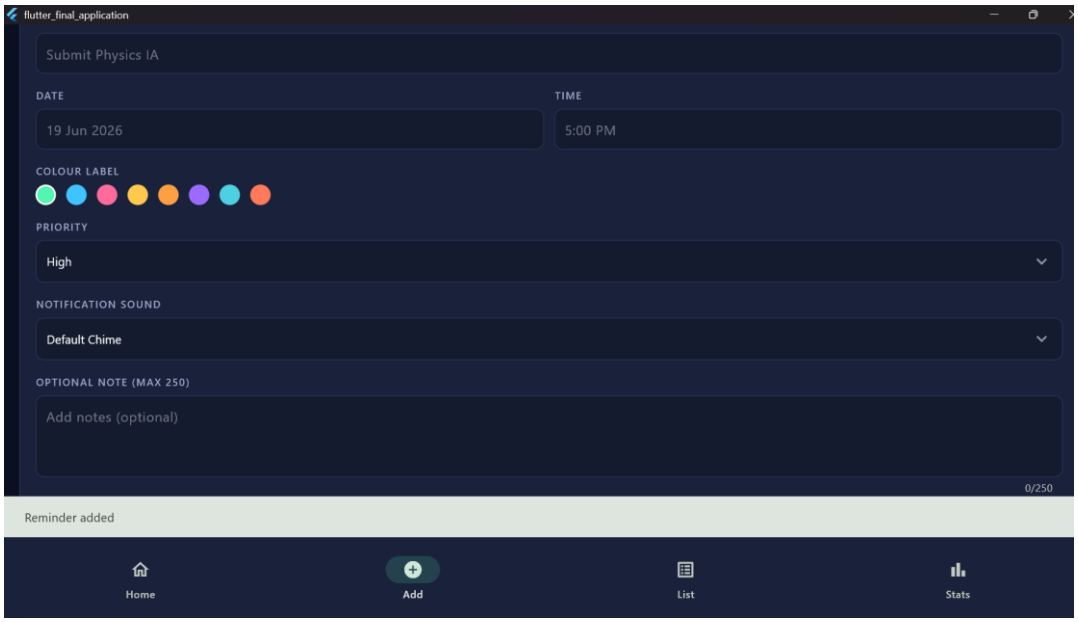


Fig. 3.1: Add Reminder Screen

```
class _AddReminderPageState extends State<AddReminderPage> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _notesController = TextEditingController();

  DateTime? _selectedDate;
  String? _selectedTime;
  String _selectedColour = '#39FF14';  // default: neon green
  String _selectedPriority = 'Medium';
  String _selectedSound = 'default';
  bool _notesExpanded = false;         // collapsible notes — CFS 2b

  @override
  void initState() {
    super.initState();
    // Pre-fill fields if editing an existing reminder — CFS 2d
    if (widget.existingReminder != null) {
      final r = widget.existingReminder!;
      _nameController.text = r.name;
      _notesController.text = r.notes ?? '';
      _selectedDate = r.date;
      _selectedTime = r.time;
      _selectedColour = r.colourHex;
      _selectedPriority = r.priority;
      _selectedSound = r.sound;
      if (r.notes != null && r.notes!.isNotEmpty) _notesExpanded = true;
    }
  }
}
```

Code Snippet 3.9: add_reminder.dart — State Variables and initState()

home.dart File

CFS 1a, 1b, 3a, 3b, 3c, 3d

The home.dart screen is the default screen displayed when FocusGrid is launched. It renders the TableCalendar widget for the current month and a filtered ListView of reminders beneath it for the selected date.

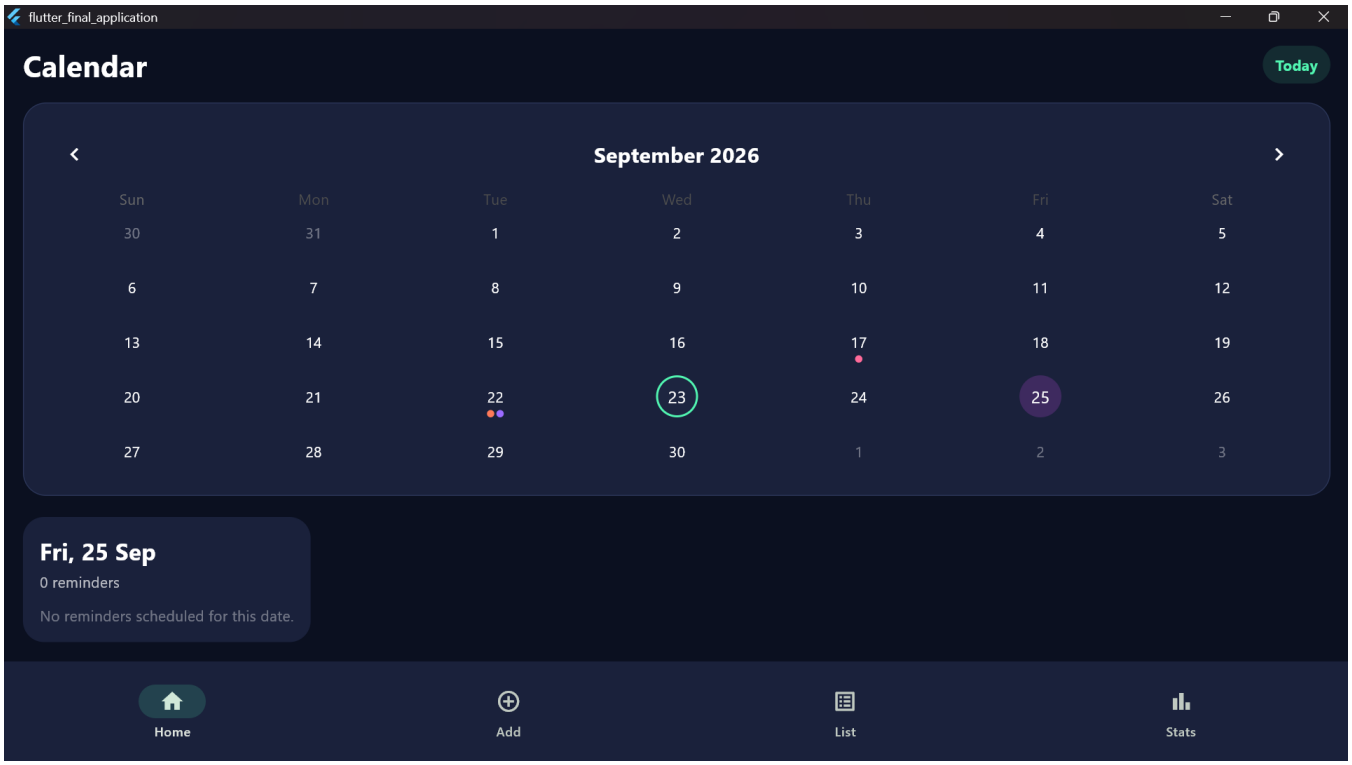


Fig. 3.2.1: Home Screen

```
class _HomePageState extends State<HomePage> {
  DateTime _focusedDay = DateTime.now();  // the visible month
  DateTime _selectedDay = DateTime.now();  // the tapped date — CFS 3c
  Map<DateTime, List<Reminder>> _remindersMap = {};

  @override
  void initState() {
    super.initState();
    _loadReminders();
  }

  // Reads all reminders from Hive and groups them by date — CFS 3b, 3d
  void _loadReminders() {
    final all = HiveHelper.instance.getAllReminders();
    final map = <DateTime, List<Reminder>>{};
    for (final r in all) {
      final key = DateTime(r.date.year, r.date.month, r.date.day);
      map.putIfAbsent(key, () => []).add(r);
    }
    setState(() => _remindersMap = map);
  }

  List<Reminder> _getEventsForDay(DateTime day) {
    return _remindersMap[DateTime(day.year, day.month, day.day)] ?? [];
  }
}
```

Code Snippet 3.11: home.dart — State and Data Loading

Code Snippet 3.12 shows the TableCalendar widget and the colour-coded dot marker builder. The markerBuilder property renders a coloured dot beneath every date that has at least one reminder, using the reminder's colourHex field.

```
// CFS 3a — full-screen interactive monthly calendar
TableCalendar<Reminder>(
  firstDay: DateTime(2020),
  lastDay: DateTime(2030),
  focusedDay: _focusedDay,
  selectedDayPredicate: (day) => isSameDay(_selectedDay, day),
  eventLoader: _getEventsForDay,        // supplies dot markers — CFS 3b
  onDaySelected: (selectedDay, focusedDay) {
    setState(() {
      _selectedDay = selectedDay;       // CFS 3c — filter list on tap
      _focusedDay = focusedDay;
    });
  },
  // CFS 3b — colour-coded dot markers below dates with reminders
  calendarBuilders: CalendarBuilders(
    markerBuilder: (context, date, events) {
      if (events.isEmpty) return const SizedBox.shrink();
      return Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: events.take(3).map((r) => Container(
          width: 6, height: 6,
          margin: const EdgeInsets.symmetric(horizontal: 1.5),
          decoration: BoxDecoration(
            shape: BoxShape.circle,
            color: Color(int.parse('0xFF${r.colourHex.replaceAll("#", "")}')),
          ),
        )).toList(),
      );
    },
  ),
)
```

Code Snippet 3.12: home.dart — TableCalendar and Colour-Coded Dot Markers

markerBuilder converts each reminder's colourHex string into a Flutter Color and draws up to three small dots beneath the date, giving an at-a-glance view of that day's reminders.

Code Snippet 3.13 shows the ListView widget that renders filtered reminder cards for the selected date directly beneath the calendar, satisfying CFS 3d.

```
// CFS 3d — displays all reminders for the selected date below the calendar
ListView.builder(
  itemCount: _getEventsForDay(_selectedDay).length,
  itemBuilder: (context, index) {
    final reminder = _getEventsForDay(_selectedDay)[index];
    return ListTile(
      leading: Container(
        width: 5,
        color: Color(int.parse('0xFF${reminder.colourHex.replaceAll("#", "")}')),
      ),
      title: Text(reminder.name),
      subtitle: Text('${reminder.time} • ${reminder.priority}'),
      onTap: () {
        Navigator.push(context, MaterialPageRoute(
          builder: (_) => AddReminderPage(existingReminder: reminder),
        )).then((_) => _loadReminders());  // reload after editing
      },
    );
  },
)
```

Code Snippet 3.13: home.dart — Daily Reminder ListView

Tapping any card navigates to AddReminderPage pre-filled with that reminder, then reloads the map on return so any edits are reflected immediately.

Feature	Purpose
_remindersMap	A Map<DateTime, List<Reminder>> grouping all Hive reminders by date. Feeds the TableCalendar eventLoader that drives dot marker rendering.
_getEventsForDay()	Looks up _remindersMap for a given date. Shared by both the TableCalendar eventLoader and the ListView builder.
markerBuilder	Renders up to three coloured dots beneath a date, coloured using each reminder's colourHex.
onDaySelected	Updates _selectedDay when the user taps a date, so the ListView shows only that date's reminders — CFS 3c, 3d.

Table 3.10: Explanation of Key Features in home.dart

reminder_list.dart File

CFS 4a, 4b, 4c, 4d

The reminder_list.dart screen displays all saved reminders across every date in a single sorted list, ascending by date then time, so the most imminent reminder always appears at the top. Each card is wrapped in a Dismissible widget for swipe-to-delete, and an edit icon navigates to the pre-filled Add Reminder screen.

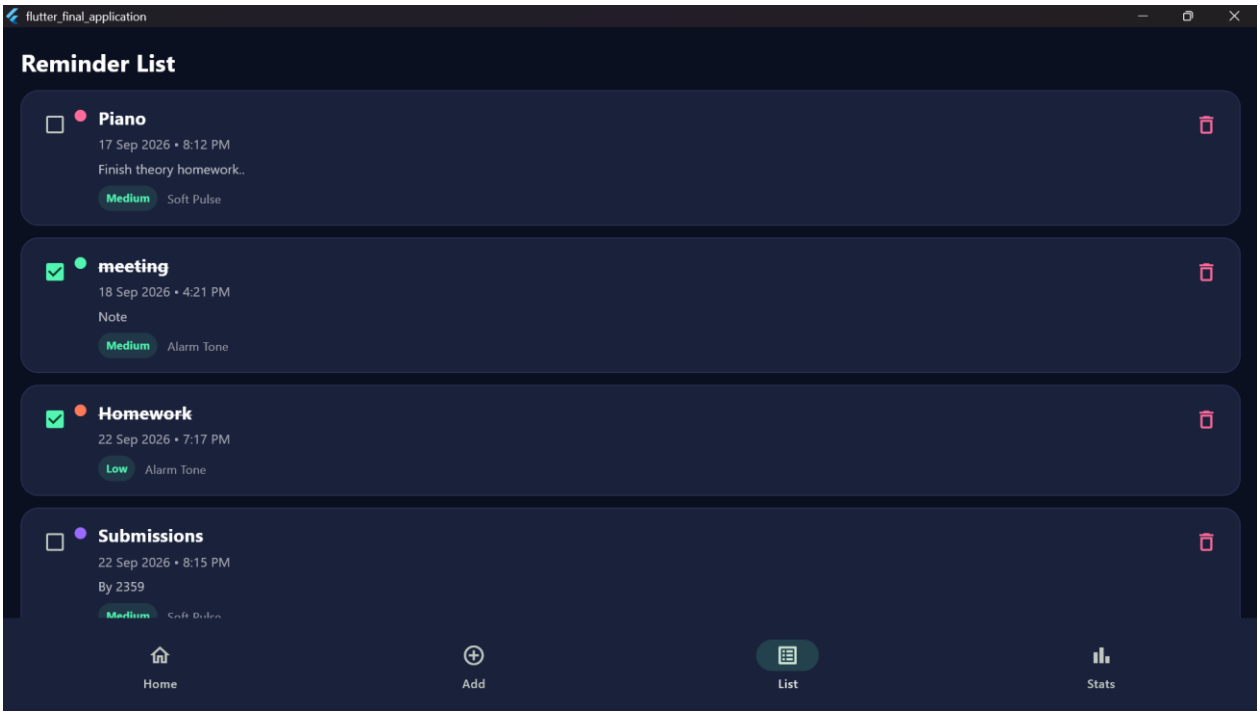


Fig. 3.3: Reminder List Screen

Code Snippet 3.15: reminder_list.dart — Dismissible Card Widget

The Dismissible widget detects an end-to-start swipe and calls _onDelete(), while the trailing edit IconButton opens AddReminderPage pre-filled and reloads the list afterwards.

Feature	Purpose
_loadSorted()	Retrieves all reminders and sorts them by date then time, re-rendering the screen via setState.
Dismissible widget	Detects a right-to-left swipe and calls _onDelete(), which removes the reminder from Hive, cancels its notification and deletes its Calendar event.
Colour bar (leading Container)	A 5dp-wide bar on the left edge of each card, coloured using the reminder's colourHex, for instant visual identification.
Priority subtitle	Combines the reminder's date, time and priority into one readable line.
Edit IconButton	Navigates to AddReminderPage pre-filled, reloading the sorted list on return.

Table 3.11: Explanation of Key Features in reminder_list.dart

AnalyticsHelper and analytics.dart File

CFS 6a, 6b

The AnalyticsHelper class contains all pure Dart logic for computing statistics from the stored Reminder dataset. Separating this logic from the UI layer means the calculations can be tested in isolation.

```
class AnalyticsHelper {
  // CFS 6a — groups all reminders by priority for the pie chart
  static Map<String, int> groupByPriority(List<Reminder> reminders) {
    final Map<String, int> counts = {'Low': 0, 'Medium': 0, 'High': 0};
    for (final r in reminders) {
      counts[r.priority] = (counts[r.priority] ?? 0) + 1;
    }
    return counts;
  }

  // CFS 6a — calculates the ratio of completed reminders as a percentage
  static double completionPercentage(List<Reminder> reminders) {
    if (reminders.isEmpty) return 0.0;
    final done = reminders.where((r) => r.isDone).length;
    return (done / reminders.length) * 100;
  }

  // CFS 6b — detects consecutive same-day gaps of 60+ minutes
  static List<FreeSlot> detectFreeSlots(List<Reminder> reminders) {
    final slots = <FreeSlot>[];
    final sorted = [...reminders]
      ..sort((a, b) {
        final d = a.date.compareTo(b.date);
        return d != 0 ? d : a.time.compareTo(b.time);
      });
    for (int i = 0; i < sorted.length - 1; i++) {
      final current = sorted[i];
      final next = sorted[i + 1];
      if (!isSameDay(current.date, next.date)) continue;
      final endCurrent = _toMinutes(current.time) + 60;
      final startNext = _toMinutes(next.time);
      final gap = startNext - endCurrent;
      if (gap >= 60) {
        slots.add(FreeSlot(date: current.date, fromMinutes: endCurrent,
            toMinutes: startNext, durationMins: gap));
      }
    }
    return slots;
  }

  static int _toMinutes(String time) {
    final p = time.split(':');
    return int.parse(p[0]) * 60 + int.parse(p[1]);
  }
}
```

Code Snippet 3.16: AnalyticsHelper — All Methods

Method	Function
groupByPriority()	Tallies the count for each priority level into a Map<String, int>, passed directly into the PieChart's data source.
completionPercentage()	Filters reminders where isDone is true and expresses the ratio as a percentage, driving the completion ring.
detectFreeSlots()	Sorts reminders and flags same-day gaps of 60+ minutes between consecutive reminders as FreeSlot objects.
_toMinutes(String time)	Converts a HH:mm string into total minutes (e.g. '09:30' → 570), used to compute gaps arithmetically.

Table 3.12: Explanation of AnalyticsHelper Methods

analytics.dart consumes AnalyticsHelper's output and displays it using the widgets in Code Snippet 3.17.

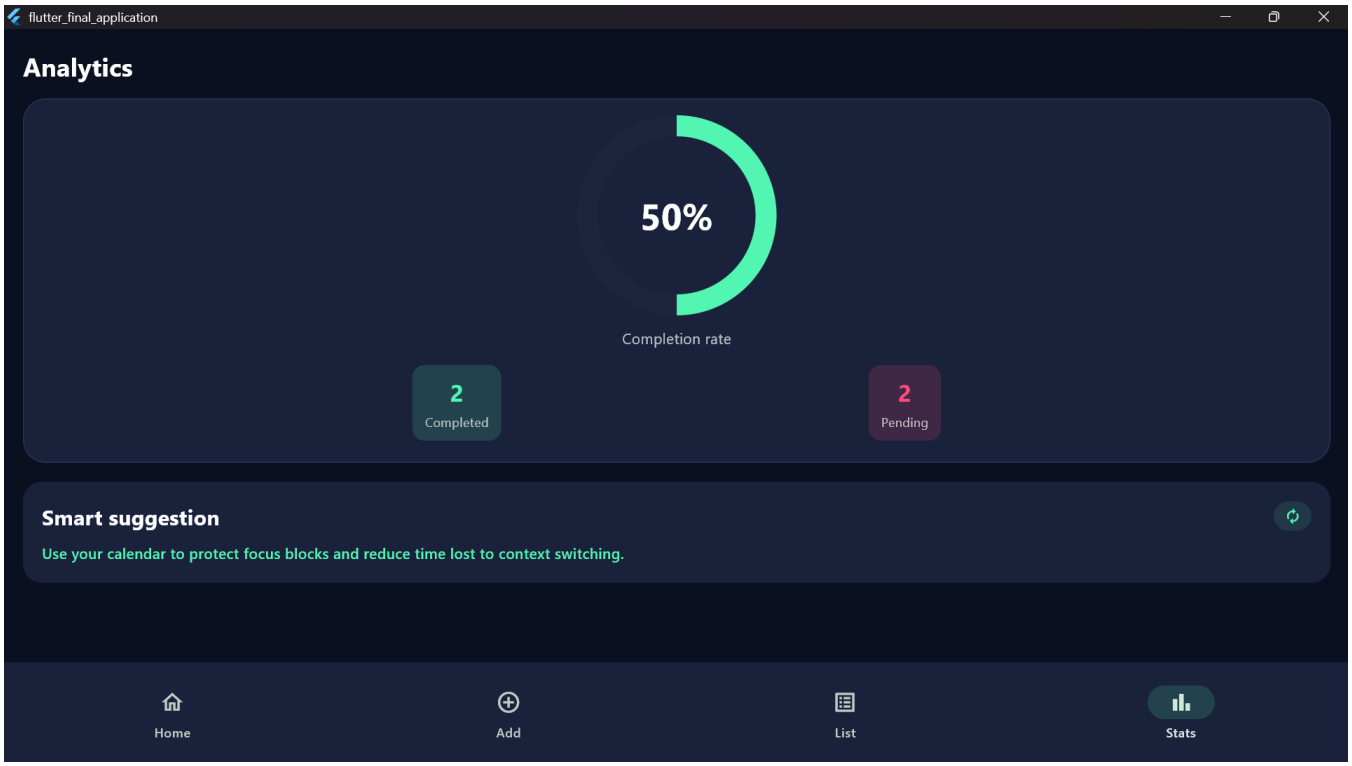


Fig. 3.4: Analytics Screen

```
void _loadAnalytics() {
  final all = HiveHelper.instance.getAllReminders();
  setState(() {
    _priorityCounts = AnalyticsHelper.groupByPriority(all);      // CFS 6a
    _completionPct = AnalyticsHelper.completionPercentage(all);  // CFS 6a
    _freeSlots = AnalyticsHelper.detectFreeSlots(all);           // CFS 6b
    _isEmpty = all.isEmpty;
  });
}

// CFS 6a — priority pie chart using fl_chart
PieChart(
  PieChartData(
    sections: _priorityCounts.entries.map((entry) => PieChartSectionData(
      value: entry.value.toDouble(),
      title: '${entry.key}\n${entry.value}',
      color: _priorityColor(entry.key),
      radius: 80,
    )).toList(),
    centerSpaceRadius: 40,
  ),
)

// CFS 6a — completion ring
CircularProgressIndicator(
  value: _completionPct / 100,
  strokeWidth: 12,
  backgroundColor: Colors.grey.shade300,
)

// CFS 6b — smart free-slot suggestion cards
Column(
  children: _freeSlots.map((slot) => Card(
    child: ListTile(
      leading: const Icon(Icons.access_time),
      title: Text('${_minsToTime(slot.fromMinutes)} – ${_minsToTime(slot.toMinutes)}'),
      subtitle: Text('${slot.durationMins} min free block'),
    ),
  )).toList(),
)
```

Code Snippet 3.17: analytics.dart — PieChart, Completion Ring and Free-Slot Cards

_loadAnalytics() recomputes all three statistics from the current dataset in one pass, and each widget reads directly from the resulting state fields.

main.dart File

main.dart is the application entry point. It initialises the timezone database, opens both Hive boxes via HiveHelper, registers the Reminder TypeAdapter, and initialises the NotificationService before calling runApp().

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeTimeZones();               // timezone database
  await Hive.initFlutter();
  await HiveHelper.instance.init();           // opens remindersBox and authBox
  await NotificationService.instance.initialize();
  runApp(const FocusGridApp());
}

class FocusGridApp extends StatelessWidget {
  const FocusGridApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'FocusGrid',
      debugShowCheckedModeBanner: false,
      home: const MainNavigation(),
    );
  }
}

class _MainNavigationState extends State<MainNavigation> {
  int _selectedIndex = 0;  // index of the currently visible screen
  static const List<Widget> _screens = [
    HomePage(), AddReminderPage(), ReminderListPage(), AnalyticsPage(),
  ];

  void _onItemTapped(int index) => setState(() => _selectedIndex = index);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens.elementAt(_selectedIndex),
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: _onItemTapped,
        type: BottomNavigationBarType.fixed,
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.calendar_today), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.add_circle), label: 'Add'),
          BottomNavigationBarItem(icon: Icon(Icons.list), label: 'List'),
          BottomNavigationBarItem(icon: Icon(Icons.bar_chart), label: 'Stats'),
        ],
      ),
    );
  }
}
```

Code Snippet 3.18: main.dart — Entry Point and Navigation Structure

_selectedIndex tracks which of the four screens is visible; _onItemTapped() updates it whenever the BottomNavigationBar is tapped, and Scaffold's body swaps to the matching screen.

Test Plan

No.	Test	Method	Expected Output	Pass?
main.dart Screen				
1.1	Verify all four BottomNavigationBar icons route to their respective screens	Tap each of the four icons in turn	The corresponding screen is displayed for each tap without error	✓
home.dart Screen				
2.1	Verify the current month's calendar is displayed on launch	Open the app and inspect the TableCalendar widget	The calendar renders the current month with today's date highlighted	✓
2.2	Verify colour-coded dot markers appear beneath dates that have reminders	Add reminders with different colours on two dates; inspect the calendar	Dots matching each reminder's colour appear below the correct dates	✓
2.3	Verify tapping a date filters the reminder list to that date only	Add reminders on two dates; tap each date in turn	Only the reminders for the tapped date appear beneath the calendar	✓
2.4	Verify tapping a reminder card opens the edit form pre-filled	Tap an existing reminder card on the home screen	Add Reminder screen opens with all fields pre-populated	✓
add_reminder.dart Screen				
3.1	Verify a new reminder can be added with all required fields	Fill in name, date, time, colour, priority and sound then press Save	Reminder appears on the calendar with a dot marker and in the list view	✓
3.2	Verify form validation rejects submission when the name field is empty	Leave the name field blank and press Save Reminder	A validation error message is shown; no reminder is saved	✓
3.3	Verify the colour swatch updates the reminder's stored colour	Select each of the six colour swatches in separate reminders and save	Each reminder card and calendar dot reflects the selected colour	✓
3.4	Verify the priority DropdownButton saves the selected priority	Select Low, Medium and High in three separate reminders	Each reminder card displays the correct priority badge	✓
3.5	Verify the collapsible notes field can be expanded and saved	Tap the notes expander, enter text, and save	The notes text is stored in Hive and shown when re-opened for editing	✓
3.6	Verify editing an existing reminder updates rather than duplicates it	Edit a saved reminder's name, date or time and save	The updated reminder replaces the original; no duplicate exists	✓
reminder_list.dart Screen				
4.1	Verify all reminders are displayed sorted by date and time ascending	Add reminders in non-chronological order and open the list screen	Reminders appear sorted with the earliest date/time at the top	✓

No.	Test	Method	Expected Output	Pass?
4.2	Verify swipe-to-delete removes the reminder from Hive and the list	Swipe a reminder card to the left	The card disappears; the reminder no longer exists in Hive	✓
4.3	Verify the edit icon on a list card opens the edit form pre-filled	Tap the edit icon on a reminder card in the list	Add Reminder screen opens pre-populated with current values	✓
Notification Service				
6.1	Verify a push notification fires at the reminder's scheduled time	Save a reminder for one minute in the future	A system notification appears with the correct title and notes body	✓
6.2	Verify deleting a reminder cancels its scheduled notification	Delete a reminder that has a future notification scheduled	The notification does not fire at the previously scheduled time	✓
analytics.dart Screen				
7.1	Verify the priority pie chart reflects the correct proportions	Add 3 Low, 2 Medium and 1 High reminder; open Analytics	Pie chart shows segments proportional to 3:2:1 with correct colours	✓
7.2	Verify the completion ring updates when a reminder is marked done	Mark one reminder as complete; observe the completion ring	The completion percentage increases immediately	✓
7.3	Verify detectFreeSlots() identifies gaps of 60+ minutes correctly	Add two reminders 90 minutes apart and one pair only 20 minutes apart	Only the 90-minute gap appears as a suggested free-slot card	✓
Data Persistence				
8.1	Verify reminder data persists after the app is fully closed and reopened	Add several reminders; fully close the app; reopen it	All previously saved reminders are still present	✓
8.2	Verify all HiveHelper CRUD methods function correctly	Use debugPrint in each method to log return values	Each method returns the expected data type and value	✓

Table 3.13: Test Plan